Deploying the Datadog Agent with Ansible

Multi-host rollout for Talend Job Servers & Remote Engines, with log collection

Deployment guide · Prepared May 2026

This guide deploys and configures the Datadog Agent across an entire fleet of Talend hosts from a single Ansible run, using the official `datadog.dd` collection (role `datadog.dd.agent`). It enables metrics, APM, process, and log collection, and registers a custom source that tails Talend's Log4j files on every host. The run is idempotent, so re-running it enforces configuration drift rather than reinstalling.

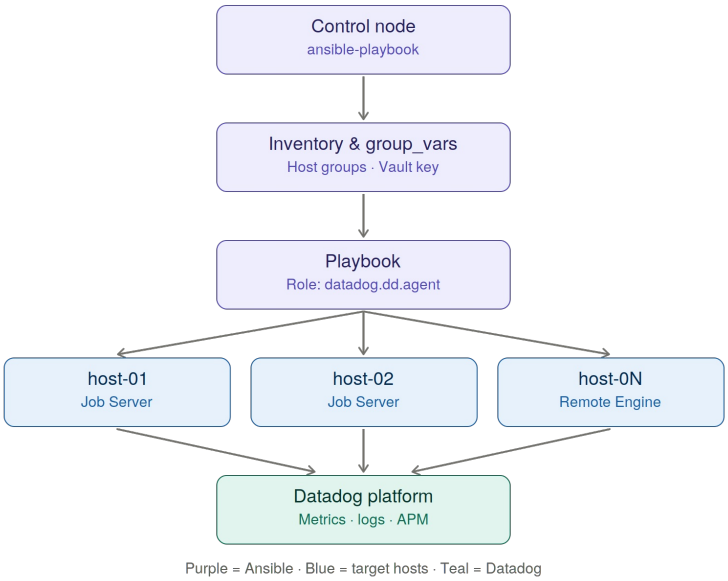

Figure 1 — The control node reads inventory and Vault-protected vars, runs a playbook that imports the role over SSH to every target host, which then forward telemetry to Datadog.

Project layout

```
datadog-deploy/
├── ansible.cfg
├── requirements.yml
├── inventory.ini
├── deploy_datadog.yml
└── group_vars/
    ├── all/
    │   └── vault.yml        # encrypted — holds the API key
    └── talend.yml           # Agent + Talend log config
```

Step 1 — Prerequisites

You need Ansible 2.10+ on the control node, SSH reachability to every target, and an account that can `sudo` on the hosts (the Agent installs system packages). The official collection is `datadog.dd` and the role inside it is `datadog.dd.agent`. The collection is tested on CentOS, Debian, Rocky Linux, openSUSE, Windows, and macOS, so a mixed fleet works.

Step 2 — Pin the collection (requirements.yml)

Declaring the dependency in a file makes the deployment reproducible across machines and CI.

```
---
collections:
  - name: datadog.dd
    version: ">=6.0.0"
```

Install it once on the control node:

```
ansible-galaxy collection install -r requirements.yml
```

Step 3 — Control-node config (ansible.cfg)

This points Ansible at the inventory, sets the SSH user, and enables fact-gathering so the role can detect each host's OS family.

```
[defaults]
inventory = ./inventory.ini
remote_user = ansible
host_key_checking = False
gathering = smart
retry_files_enabled = False
stdout_callback = yaml

[privilege_escalation]
become = True
become_method = sudo
```

Step 4 — Inventory (inventory.ini)

Group hosts so the playbook targets them as a unit. All Talend boxes live under `[talend]`, split into sub-groups so you could apply different vars later, but they all inherit the group config.

```
[talend_jobservers]
host-01 ansible_host=10.0.1.11
host-02 ansible_host=10.0.1.12

[talend_remote_engines]
host-03 ansible_host=10.0.1.21

[talend:children]
talend_jobservers
talend_remote_engines
```

Because the group is named `talend`, Ansible auto-loads `group_vars/talend.yml` for every host in it.

Step 5 — Protect the API key with Vault

Never store the API key in plaintext. Create an encrypted file:

```
ansible-vault create group_vars/all/vault.yml
```

Inside, define one variable:

```
---
vault_datadog_api_key: "your-real-32-char-datadog-api-key"
```

Anything under `group_vars/all/` loads for every host, so the key is available everywhere but stays encrypted at rest.

Step 6 — Agent + Talend config (group_vars/talend.yml)

This is the heart of the deployment. It selects the Agent version, turns on logs/APM/process collection, applies unified tags, and registers a custom `talend` log source that tails the Log4j files fleet-wide.

```
---
datadog_api_key: "{{ vault_datadog_api_key }}"
datadog_site: "datadoghq.com"
datadog_agent_major_version: 7

datadog_config:
  log_level: INFO
  logs_enabled: true
  apm_config:
    enabled: true
  process_config:
    process_collection:
      enabled: true
  tags:
    - "env:prod"
    - "service:talend-etl"
    - "team:data-eng"

# Each key becomes conf.d/<key>.d/conf.yaml on the host.
datadog_checks:
  talend:
    logs:
      - type: file
        path: /opt/talend/jobs/*/logs/*.log
        source: talend
        service: talend-etl
        log_processing_rules:
          - type: multi_line
            name: collapse_java_stack_traces
            pattern: \d{4}-\d{2}-\d{2}
```

`datadog_config` maps directly to `datadog.yaml` on each host, so `logs_enabled` flips on the Log Agent globally. `datadog_checks` drives per-integration config — the role writes each key as `conf.d/<key>.d/conf.yaml`, so the `talend` block lands at `/etc/datadog-agent/conf.d/talend.d/conf.yaml`. The multi-line rule keeps Java stack traces as single events, and the shared `service` tag ties these logs to the same service as the metrics so they correlate on dashboards.

Step 7 — The playbook (deploy_datadog.yml)

The playbook is deliberately thin: validate the key exists, import the role, then confirm the service is up. All configuration lives in vars, which keeps it reusable across environments.

```
---
- name: Deploy and configure the Datadog Agent on Talend hosts
  hosts: talend
  become: true

  pre_tasks:
    - name: Fail fast if the API key is missing
      ansible.builtin.assert:
        that:
          - datadog_api_key is defined
          - datadog_api_key | length > 20
        fail_msg: "vault_datadog_api_key is not set or looks invalid."

  roles:
    - role: datadog.dd.agent

  post_tasks:
    - name: Gather service facts
      ansible.builtin.service_facts:

    - name: Assert datadog-agent is active
      ansible.builtin.assert:
        that:
          - "'datadog-agent.service' in ansible_facts.services"
          - "ansible_facts.services['datadog-agent.service'].state == 'running'"
        fail_msg: "datadog-agent is not running on {{ inventory_hostname }}."
```

Step 8 — Run it

Dry-run first to preview changes without touching anything:

```
ansible-playbook deploy_datadog.yml --ask-vault-pass --check --diff
```

Then apply for real. Roll out gradually with `--limit`:

```
# canary one host first
ansible-playbook deploy_datadog.yml --ask-vault-pass --limit host-01

# then the whole fleet
ansible-playbook deploy_datadog.yml --ask-vault-pass
```

`--ask-vault-pass` prompts for the Vault password so the key is decrypted at runtime; in CI, use `--vault-password-file` pointing at a secret instead.

Step 9 — Validate

The playbook's `post_tasks` already assert the service is up. To confirm end to end, SSH to any host and run `sudo datadog-agent status` (look for the Logs Agent reporting the `talend` source). Within a minute or two the hosts appear in Datadog under Infrastructure → Host Map tagged `service:talend-etl`, and `source:talend` logs flow into the Log Explorer.

> **Before production:** adjust the log `path` glob to match your Talend install (it differs between Job Server and Remote Engine), and set `tags` / `env` per environment if you split prod and non-prod inventories. Re-running the playbook is safe and only corrects drift.